OPENSTAGE

Manuale tecnico e guida utente

Manuale tecnico e guida utente

Versione: 1.0

Data: 16 ottobre 2026

Contenuto: Fase 1 · Fase 1.2 · Fase 1.3

Documento generato automaticamente dai sorgenti in docs/manuale. Non modificare il PDF a mano: si aggiorna con npm run manuale.

Indice

1. Che cos'è OpenStage

- A che punto siamo

2. I ruoli e le loro regole

- La regola importante: il ruolo è definitivo

3. Registrazione e primo accesso

- Con email e password
- Con Google
- La barra in alto quando sei dentro
- Diventare artista o locale (solo se sei un fan)

4. Navigare il sito

- Lingua
- Il player in basso

5. Domande frequenti

6. Architettura e stack

- Alternative valutate e perché sono state scartate
- Versioni dell'ambiente

7. Requisiti, comandi e struttura del progetto

- Struttura delle cartelle

8. Variabili d'ambiente

9. Autenticazione e gestione del ruolo

- Come funziona l'accesso
- Dove sono applicate le regole del ruolo

10. Database

- Tabelle
- Funzioni e trigger
- Indici
- Migrazioni

11. Sicurezza: Row Level Security

- Avvisi di sicurezza attualmente accettati

12. Come si collauda la RLS

13. Internazionalizzazione

14. Design system

15. Deploy e rilascio

- Checklist di fine fase

16. Errori già incontrati e come si risolvono

17. Il manuale PDF

18. Roadmap delle fasi

19. Diario delle fasi

- Fase 1 — Fondamenta
- Fase 1.2 — Indurimento del database
- Fase 1.3 — Ruoli definitivi
- Come si aggiorna questo diario

20. Changelog del manuale

- Come si aggiorna

> OpenStage · Manuale tecnico e guida utente · versione 1.0 (16 ottobre 2026)
> Il PDF viene rigenerato dai file Markdown in `docs/manuale/` con `npm run manuale`.

Che cos'è OpenStage

OpenStage è una piattaforma per **scoprire concerti dal vivo**, pensata per dare visibilità reale agli **artisti indipendenti** e ai **locali di zona** che non possono permettersi le grandi piattaforme commerciali.

I quattro principi che guidano ogni decisione (anche tecnica) sono:

Principio	Cosa significa in pratica
Underground e indipendente	Gli artisti emergenti e i piccoli locali sono i protagonisti, non un contorno
Gratis per sempre	Nessun paywall per fan, artisti e locali nel piano base, nessun "pay-to-play"
Autenticità	Eventi reali, luoghi reali: niente metriche inventate o engagement finto
UX senza attrito	Trovare il concerto di stasera dev'essere facile come aprire una mappa

> **Stato attuale.** Il progetto è in costruzione per fasi. Questo manuale documenta tutto ciò che è **realmente online e funzionante** alla versione indicata in copertina: quando una funzione è ancora un abbozzo, è scritto chiaramente.

A che punto siamo

Fatto e online: identità e accesso (email e Google), ruoli con regole definitive, design system, doppia lingua IT/EN, pagine pubbliche (Home, Artisti, Locali, Per gli Artisti, Per le Venue, Chi siamo, Contatti, Privacy, Termini, Cookie), database con sicurezza a livello di riga, deploy continuo.

In arrivo nelle prossime fasi: mappa interattiva degli eventi, pubblicazione e gestione degli eventi, schede artista e locale, ricerca e filtri, notifiche email, pagamenti.

I ruoli e le loro regole

Quando ti registri scegli **come vuoi usare OpenStage**. La scelta non è un'etichetta cosmetica: determina cosa puoi fare e quali pagine vedi.

Ruolo	Per chi è	Cosa può fare
Fan	Chi vuole scoprire concerti	Esplorare artisti, locali ed eventi; salvare i preferiti (prossime fasi)
Artista	Musicisti, band, dj, progetti	Tutto ciò che fa un fan, più: profilo artista e pubblicazione dei propri concerti
Locale / Organizzatore	Locali, circoli, festival, promoter	Tutto ciò che fa un fan, più: profilo del locale e programmazione degli eventi
Admin	Solo lo staff di OpenStage	Accesso completo per assistenza e correzioni

La regola importante: il ruolo è definitivo

Questa è una scelta precisa del progetto, e vale **sia per la registrazione via email sia per quella con Google**:

Situazione	Cosa succede
Ti registri come **Artista**	Resti artista **per sempre**
Ti registri come **Locale**	Resti locale **per sempre**
Ti registri come **Fan**	Puoi diventare **Artista o Locale**, ma **una volta sola**; dopo non si torna indietro
Sei un fan e vuoi tornare fan dopo essere diventato artista	Ti serve un **nuovo account**

Il motivo è la coerenza della community: un profilo artista non deve poter diventare un locale (o viceversa) dall'oggi al domani, perché gli eventi, i fan e i locali che ci hanno interagito si fidano di quel profilo.

> **Se hai sbagliato a scegliere**, scrivi allo staff: l'admin può correggere il ruolo dalla dashboard (è l'unica eccezione prevista alla regola).

Registrazione e primo accesso

Con email e password

1. Vai su **Registrati** dalla barra in alto.
2. Inserisci nome, email, password e scegli il ruolo (Fan, Artista, Locale).
3. Conferma il **captcha** (protezione anti-bot Cloudflare Turnstile).
4. Sei dentro. Se hai scelto **Artista**, vieni portato direttamente alla compilazione del profilo artista; altrimenti torni alla home.

> In questa fase la **conferma via email è disattivata**: l'account è attivo subito. Nelle fasi successive verrà attivata con le email transazionali.

Con Google

1. Clicca **Continua con Google** nella pagina di accesso o registrazione.
2. Scegli il tuo account Google.
3. Google **non** trasmette a OpenStage il ruolo scelto prima dell'accesso: per questo, al primo ingresso, compare una **schermata di scelta del ruolo**. Da quel momento la regola dei ruoli definitivi vale esattamente come per l'email.

La barra in alto quando sei dentro

Dopo l'accesso, in alto a destra vedi:

- il tuo **indirizzo email**;
- un **badge colorato con il tuo ruolo** (viola = Artista, ciano = Locale, rosa = Admin, grigio = Fan);
- un **link contestuale**: per un fan "Diventa artista o locale", per un artista o un locale "Crea evento";
- il pulsante **Esci**.

Diventare artista o locale (solo se sei un fan)

1. Clicca **Diventa artista o locale** nella barra in alto.
2. Scegli **Artista** o **Locale** e conferma.
3. Se scegli Artista vieni portato al profilo artista; il badge in alto cambia subito colore e il link di upgrade **sparisce**, perché la scelta è definitiva.

Se torni su quella pagina dopo la scelta, OpenStage ti mostra **"Il tuo ruolo è già attivo"** con il tuo ruolo: è la conferma che il blocco funziona anche lato database.

Navigare il sito

Voce	Cosa trovi
Esplora (Home)	La vetrina: hero animato, artisti e locali in evidenza, invito a cercare
Artisti	Elenco dei profili artista pubblicati, con paginazione
Locali	Elenco dei locali e degli organizzatori
Per gli Artisti	Perché OpenStage conviene a chi suona: visibilità, nessun costo, contatto diretto
Per le Venue	Come i locali riempiono la sala e collaborano con gli artisti di zona
Chi siamo	La storia e i valori del progetto
Accedi / Registrati	Accesso e creazione account

In fondo alla pagina trovi il **footer** con contatti, Privacy Policy, Termini di Servizio, Cookie Policy e i link social.

Lingua

La piattaforma parte in **italiano** se ti colleghi dall'Italia, in **inglese** da qualsiasi altro paese. Puoi cambiarla quando vuoi con il selettore **IT / EN** nella barra in alto: la scelta viene ricordata anche alla visita successiva.

Il player in basso

In fondo allo schermo c'è un **player Spotify** fisso: serve per ascoltare in background le playlist di OpenStage mentre navighi, senza interrompere la navigazione (anche su mobile, sopra il menu).

Domande frequenti

Posso avere due profili (uno da fan e uno da artista) con la stessa email? No: ogni email è un account. Se hai scelto artista e volevi fare il fan, puoi comunque esplorare tutto e salvare i preferiti: le funzioni da fan restano disponibili.

Ho scelto Locale ma sono un musicista: posso cambiare? No, il ruolo è definitivo. Scrivi allo staff: solo l'admin può correggere.

Perché mi ha chiesto il ruolo dopo l'accesso con Google? Perché Google non può trasportare quell'informazione. È il passaggio in più che sostituisce la scelta fatta al momento della registrazione via email.

Devo pagare qualcosa? No. Il piano base è gratuito per fan, artisti e locali: è un principio del progetto.

Non vedo ancora la mappa o la creazione degli eventi: sono rotte? No, non sono ancora state costruite: arrivano nelle prossime fasi. Se provi ad aprire "Crea evento" oggi vedi una pagina segnaposto.

Architettura e stack

OpenStage è una **applicazione web Next.js** (App Router, TypeScript) renderizzata **lato server** su tutte le pagine pubbliche: ogni pagina arriva già composta, così SEO e anteprime social funzionano sempre e non esiste una "SPA" vuota da riempire nel browser.

Pezzo	Tecnologia	Versione	Perché
Framework	Next.js (App Router)	15.5.28	SSR ovunque, server actions, ottimo su Vercel
UI	React	19.1.0	Richiesto da Next 15
Linguaggio	TypeScript	5.x	Errori presi prima del deploy
Stile	Tailwind CSS	v4	Utility-first, tema custom con i token del brand
Animazioni	Framer Motion	*in arrivo*	Scroll-reveal, tilt 3D, parallax (fase design avanzato)
Database + Auth	Supabase (Postgres)	eu-north-1	RLS nativa sull'`auth.uid()`; un solo fornitore per dati e login
Anti-bot	Cloudflare Turnstile	—	Captcha invisibile su registrazione e login
Mappe	MapLibre GL + MapTiler	*in arrivo*	Mappa degli eventi (Fase 2)
Email	Resend	*in arrivo*	Email transazionali (fasi successive)
Pagamenti	PayPal	*in arrivo*	Solo funzioni opzionali: il piano base resta gratuito
Hosting	Vercel	Hobby	Deploy continuo da GitHub
Repository	GitHub	`openstagemusic/openstage`	Ramo principale `main`

Alternative valutate e perché sono state scartate

Alternativa	Motivo del no
Neon + Clerk	Due fornitori da configurare, e Clerk non ha RLS sul database: la sicurezza dei dati va riscritta a mano nell'app
Nhost / Appwrite / Firebase	Livello database meno "Postgres puro" (o NoSQL): vincoli, viste e policy SQL perdono potenza
Turso	SQLite distribuito: ottimo per altri casi, ma qui serve Postgres con RLS nativa
Supabase	**Scelto**: Postgres con Row Level Security collegata all'utente loggato, Auth integrata, piano gratuito ampio

Versioni dell'ambiente

- **Node.js**: 24.x (in locale 24.16.0, su Vercel `nodeVersion` 24.x)
- **npm**: 11.x
- **Regione Supabase**: `eu-north-1` (dati in Europa)
- **Progetto Supabase (ref)**: `tzdigzvkneonkhkwoeed`

Requisiti, comandi e struttura del progetto

Comando	Cosa fa
`npm install`	Installa le dipendenze
`npm run dev`	Avvia lo sviluppo locale su `http://localhost:3000` (Turbopack)
`npm run build`	**Build di produzione**: va eseguita a ogni fine fase, senza errori
`npm run start`	Avvia la build già compilata
`npm run lint`	Controlla il codice con ESLint
`npm run manuale`	Rigenera **questo PDF** dai sorgenti in `docs/manuale/`

Per allineare le variabili d'ambiente locali a quelle di Vercel: `npx vercel env pull .env.local`.

Struttura delle cartelle

```
app/                      Pagine (App Router) e server action
  layout.tsx              Navbar, footer, player, lingua: struttura di ogni pagina
  page.tsx                Home (Esplora)
  artisti/ locali/        Elenchi pubblici (leggono dal database, con paginazione)
  per-gli-artisti/ per-le-venue/ about/ contatti/ privacy/ terms/ cookies/
  login/ register/        Accesso e registrazione
  auth/callback/          Ritorno da Google: crea/aggiorna la sessione
  onboarding/role/        Scelta del ruolo ("Diventa artista o locale")
  onboarding/artist/      Profilo artista
  events/new/             Creazione evento (segnaposto: arriva in Fase 2)
  actions/locale.ts       Server action che salva la lingua scelta
components/
  layout/                 Navbar, Footer, SpotifyPlayerDock
  auth/                   RegisterForm, LoginForm, GoogleButton, RolePicker,
                          OnboardingForm, TurnstileWidget, LogoutButton
  ui/                     Pagination, PagePlaceholder
  i18n/                   LanguageSwitcher
i18n/dictionaries/        it.ts e en.ts: tutti i testi dell'interfaccia
lib/
  auth.ts                 isAdminEmail() per le decisioni di interfaccia
  auth/roles.ts           Tipo UserRole, etichette, permessi (eventi, profilo artista)
  i18n/                   config (cookie/lingue), server (lettura), metadata, dictionaries
  supabase/               client, server, admin, env, middleware, session
supabase/migrations/      0001_init, 0002_role_self_service, 0003_hardening, 0004_role_lock
docs/                     Piani di fase, review di fase, sorgenti del manuale
scripts/manuale.mjs       Generatore del PDF (solo locale, non va in produzione)
```

`middleware.ts` (nella radice) rinnova la sessione Supabase a ogni richiesta e, alla prima visita, sceglie la lingua in base al paese rilevato da Vercel.

Variabili d'ambiente

Le stesse variabili esistono in **tre ambienti** su Vercel (Production, Preview, Development): quando ne aggiungi una, impostala su tutti e tre.

Variabile	Dove si prende	A cosa serve
`NEXT_PUBLIC_SUPABASE_URL`	Supabase → Project Settings → API	Indirizzo del progetto Supabase
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase → Project Settings → API	Chiave pubblica (protetta dalla RLS)
`SUPABASE_SERVICE_ROLE_KEY`	Supabase → Project Settings → API	Chiave di servizio: **solo server**, da usare con cautela. Oggi presente ma non ancora usata
`NEXT_PUBLIC_TURNSTILE_SITE_KEY`	Cloudflare → Turnstile	Chiave pubblica del captcha
`TURNSTILE_SECRET_KEY`	Cloudflare → Turnstile	Verifica lato server (Supabase la usa per il captcha)
`ADMIN_EMAIL`	Scelta interna	Email dello staff: deve combaciare con `settings.admin_email`
`NEXT_PUBLIC_SITE_URL`	Dominio di produzione	URL canonico per SEO e link di ritorno
`NEXT_PUBLIC_SPOTIFY_PLAYLIST_ID`	URL della playlist	ID della playlist del player in basso

> `.env.local` **non deve mai** finire su Git: il file è già escluso. Il modello da copiare è `.env.example`. Due variabili sono marcate **sensitive** su Vercel (`SUPABASE_SERVICE_ROLE_KEY`, `TURNSTILE_SECRET_KEY`): una volta salvate non sono più leggibili, per questo è normale non poterle "rivedere" dalla dashboard.

Autenticazione e gestione del ruolo

Come funziona l'accesso

```
Email + password:  form → Supabase Auth → trigger handle_new_user() crea la riga in
profiles
                                        → il ruolo arriva dai metadati del form
(fan|artist|venue)

Google:            bottone → Google → /auth/callback → sessione
                                        → se manca il ruolo, si passa da /onboarding/role
```

Punti importanti:

1. **Il database è la fonte di verità del ruolo.** Il ruolo vive in `public.profiles.role`; il badge in navbar e le pagine protette leggono da lì (helper `getSessionProfile()`, con `cache()` di React: una sola lettura per richiesta, condivisa tra layout e pagina).
2. **Google non trasporta il ruolo.** Non lo si passa nell'URL di redirect perché sarebbe un parametro che il browser può manipolare. Perciò, al primo ingresso con Google, il ruolo si sceglie in una pagina dedicata e viene salvato sia nel profilo sia nei metadati (`user_metadata.role`), che servono come marcatore.

Dove sono applicate le regole del ruolo

Le regole valgono su **tre livelli**, dal più importante al più visibile:

Livello	Cosa fa	Dove
Database	Rifiuta le transizioni di ruolo non ammesse e le righe incoerenti	Trigger `prevent_role_escalation` + policy RLS
Pagine	Un fan non entra nelle pagine da artista, e viene indirizzato alla scelta del ruolo	`app/onboarding/artist`, `app/events/new`, `app/onboarding/role`
Interfaccia	Mostra il ruolo e le azioni giuste (badge + link contestuale)	`components/layout/Navbar.tsx`, `lib/auth/roles.ts`

Regola delle transizioni, in una riga: **solo `fan → artista | locale`**, una volta sola. Tutto il resto viene annullato. L'admin può correggere (eccezione voluta, e anche le modifiche fatte direttamente in SQL dalla dashboard non vengono bloccate).

Database

Tabelle

`profiles` — una riga per utente (creata dal trigger alla registrazione). `id` (chiave, uguale a `auth.users.id`), `email`, `full_name`, `role` (**enum**: `fan` / `artist` / `venue` / `admin`, predefinito `fan`), `city`, `created_at`, `updated_at`.

`artists` — profilo pubblico dell'artista. `id`, `profile_id` (unico → un solo profilo artista per utente), `name`, `city`, `genre`, `spotify_url`, `bio`, `created_at`, `updated_at`.

`venues` — profilo pubblico del locale/organizzatore. `id`, `profile_id` (unico), `name`, `city`, `address`, `latitude`, `longitude`, `website`, `created_at`, `updated_at`.

`events` — gli eventi. `id`, `title`, `description`, `artist_id`, `venue_id`, `starts_at`, `city`, `is_unclaimed` (evento inserito da OpenStage e non ancora rivendicato dal locale), `status` (**enum**: `draft` / `published` / `cancelled`), `created_by`, `created_at`, `updated_at`.

`settings` — parametri interni chiave/valore. Oggi contiene `admin_email` = email dello staff. È leggibile solo dall'admin.

Funzioni e trigger

Oggetto	Cosa fa
`handle_new_user()`	Alla registrazione crea la riga in `profiles` e assegna il ruolo (o `admin` se l'email è quella dello staff)
`prevent_role_escalation()`	Blocca le transizioni di ruolo non ammesse (vedi sopra)
`touch_updated_at()`	Aggiorna `updated_at` a ogni modifica
`is_admin()`	Vero se l'utente loggato è l'email in `settings.admin_email`. Usata dalle policy
Trigger	`on_auth_user_created` (su `auth.users`), `profiles_guard`, `artists_touch`, `venues_touch`, `events_touch`, `settings_touch`

Indici

Chiavi primarie e unicità su `profiles.id`, `artists.profile_id`, `venues.profile_id`, `settings.key`, più gli indici per le ricerche frequenti: `events_starts_at_idx` (ordine per data), `events_city_idx` (filtro per città), `events_artist_id_idx`, `events_venue_id_idx`, `events_created_by_idx` (i collegamenti tra tabelle).

Migrazioni

Le migrazioni stanno in `supabase/migrations/` e vanno applicate **in ordine**, una volta sola:

File	Contenuto
`0001_init.sql`	Schema iniziale: tabelle, enum, policy, trigger, impostazioni
`0002_role_self_service.sql`	Il ruolo `fan` / `artista` / `locale` è autodichiarabile (l'admin no)
`0003_hardening.sql`	Indurimento: `search_path`, revoche, indici, policy riscritte con `(select auth.uid())`
`0004_role_lock.sql`	Blocco delle transizioni di ruolo + coerenza del ruolo nella RLS

> Lo storico ufficiale è nella tabella `supabase_migrations.schema_migrations` (voci `0001` … `0004`). Se si aggiunge una migrazione, va aggiunta **anche** quella riga, altrimenti `supabase migration list` non la vede.

Sicurezza: Row Level Security

Tutte le tabelle hanno la **RLS attiva**: senza una policy che lo consenta, una riga non è visibile né modificabile. Regola d'oro: le policy parlano di `auth.uid()` (l'utente loggato) e di `is_admin()`, mai di valori passati dal browser.

Tabella	Policy	Regola in parole semplici
`profiles`	`profiles_select_own_or_admin`	Vedi solo il tuo profilo (l'admin tutti)
	`profiles_update_own_or_admin`	Modifichi solo il tuo (l'admin tutti) — il ruolo passa dal trigger
`artists`	`artists_select_public`	Le schede artista sono pubbliche
	`artists_insert_own`	Crei la tua riga **solo se il tuo ruolo è** `artist`
	`artists_update_own`, `artists_delete_own`	Gestisci solo la tua riga (o l'admin)
`venues`	`venues_select_public`	Le schede locale sono pubbliche
	`venues_insert_own`	Crei la tua riga **solo se il tuo ruolo è** `venue`
	`venues_update_own`, `venues_delete_own`	Gestisci solo la tua riga (o l'admin)
`events`	`events_select_public`	Gli eventi sono pubblici
	`events_insert_authenticated`	Crei un evento **solo se sei** `artist` **o** `venue` e come tuo
	`events_update_own`, `events_delete_own`	Modifichi solo i tuoi eventi (o l'admin)
`settings`	`settings_select_admin`, `settings_update_admin`	Solo l'admin

Perché le policy contano davvero: un utente malintenzionato può chiamare l'API di Supabase direttamente, senza passare dall'interfaccia. Le regole "il fan non vede il pulsante" valgono solo per l'interfaccia; quelle che contano sono nel database. Per esempio, un fan che prova a creare una riga artista via API riceve un **errore 42501 (row-level security policy)**, anche se la pagina web non gliela mostrerebbe mai.

Avvisi di sicurezza attualmente accettati

Gli strumenti di Supabase segnalano ogni tanto dei punti da valutare. Oggi restano aperti, con motivazione:

Avviso	Perché è accettato
`is_admin()` è `SECURITY DEFINER` ed eseguibile da anonymous/authenticated	Le policy RLS la invocano: deve poter leggere `settings` (che è riservata all'admin) senza esporre nient'altro
Leaked password protection disattivata	Da attivare in Supabase → Authentication → Settings (può richiedere il piano a pagamento)
Indici "mai usati" su `events`	La tabella è ancora vuota: diventeranno utili con gli eventi reali (Fase 2)

Come si collauda la RLS

Non serve un browser: si impersona un utente direttamente in SQL e si legge il risultato. Con gli strumenti MCP di Supabase funziona così:

```
-- 1. Ci si mette nei panni di un utente loggato…
set local role authenticated;
set local request.jwt.claims = '{"sub":"<uuid-utente>","email":"
<email>","role":"authenticated"}';

-- 2. …si prova l'operazione. Se la RLS non la consente, arriva un errore 42501.
update public.profiles set role = 'venue' where id = '<uuid-utente>';

-- 3. Si torna a leggere come admin per verificare l'effetto reale.
reset role;
select role from public.profiles where id = '<uuid-utente>';
```

Due accortezze imparate sul campo:

1. **I claim restano per tutta la transazione.** Se nella stessa sessione provi dopo una modifica "da dashboard", la GUC dei claim è ancora attiva e il test mente. Azzerala con `set local request.jwt.claims = ''` prima di simulare l'SQL diretto.
2. Per catturare l'esito di un'operazione che deve **fallire**, avvolgila in un blocco `do $$ … exception when others then … $$` e salva l'esito con `set_config(..., true)`, così l'errore non interrompe tutto il resto del collaudo.

Vale la pena farlo per: transizioni di ruolo, insert in `artists` / `venues` / `events` con ruolo sbagliato, visibilità dei profili altrui.

Internazionalizzazione

- Due dizionari: `i18n/dictionaries/it.ts` e `i18n/dictionaries/en.ts`.
- **`en.ts` è la fonte dei tipi**: se aggiungi una chiave solo in `it.ts`, la build fallisce. Aggiungere un testo significa quindi **sempre** toccare entrambi i file, con lo stesso nome.
- Lingua iniziale: l'italiano se il visitatore arriva dall'Italia (`x-vercel-ip-country`), altrimenti l'inglese. La scelta manuale (switcher IT/EN) viene salvata nel cookie `NEXT_LOCALE` e da quel momento ha la precedenza.
- Le pagine di supporto (Privacy, Termini, Cookie, Chi siamo, Contatti) sono bilingue come tutto il resto.

Design system

Elemento	Valore
Sfondo	`#07070D`
Viola (primario)	`#8B5CF6`
Ciano (secondario)	`#22D3EE`
Rosa (accento)	`#F472B6`
Titoli	Space Grotesk
Corpo del testo	Inter
Effetti	Glass (blur + bordo traslucido 1px), gradienti viola→ciano, glow

Regole da non tradire:

1. **Glass solo su navbar, card e pop-up**: mai dietro a testi lunghi, altrimenti la leggibilità crolla.
2. **Mobile first**: si progetta prima lo schermo piccolo.
3. I token del tema stanno in `app/globals.css`: cambiando lì i colori, cambiano ovunque.

Deploy e rilascio

1. Il codice vive su GitHub (`openstagemusic/openstage`, ramo `main`).
2. Ogni push su `main` fa partire automaticamente un **deploy di produzione** su Vercel.
3. Progetto Vercel: `openstage` (team `ceskobeatzmusic-2476`), piano Hobby.
4. **Il preset del framework deve restare "Next.js"**: una volta è stato impostato su "Other" e i deploy si rompevano. Non va mai reintrodotto quell'override.
5. La protezione "Vercel Authentication" (Deployment Protection) è **disattivata**, altrimenti il sito pubblico chiederebbe un login Vercel.

URL di produzione: `https://openstage-alpha.vercel.app`

Checklist di fine fase

1. `npm run build` senza errori.
2. Verifica delle regole nel database (SQL di collaudo qui sopra).
3. Verifica delle pagine in produzione, anche da utente non loggato (i percorsi protetti devono rimandare al login).
4. Commit su Git chiamato **"Fase N"** e push.
5. `docs/review-fase-N.md` aggiornato (cosa funziona, suggerimenti, debito tecnico, cose da riprendere).
6. Se la fase ha cambiato qualcosa di stabile (architettura, variabili, regole, schermate): `npm run manuale` per rigenerare questo PDF, con versione e data nuove in `meta.json`.

Errori già incontrati e come si risolvono

Sintomo	Causa	Rimedio
Il deploy su Vercel si rompe senza un errore di codice	Preset del framework impostato su "Other"	Riportare il preset a **Next.js** e ricreare il deploy di produzione
In locale la build va, su Vercel no (o viceversa)	Variabile d'ambiente mancante in uno dei tre ambienti	Impostarla su Production, Preview e Development
Login/registrazione dall'API falliscono con errore captcha	Turnstile è attivo su Supabase: serve un token vero	Provare da browser, non con chiamate dirette
`supabase migration list` non mostra le migrazioni	Manca la riga in `supabase_migrations.schema_migrations`	Inserire la voce `(version, name)` corrispondente al file
Un test RLS "mente" (una modifica bloccata sembra passata)	I claim JWT erano rimasti attivi da un test precedente	`set local request.jwt.claims = ''` prima del test
L'URL di autorizzazione OAuth arriva rotto (`client_id: invalid`)	I parametri `&` vengono corrotti aprendolo da terminale/chat	Incollare l'URL a mano nella barra degli indirizzi
Il selettore di lingua non cambia nulla	La scelta manuale è nel cookie `NEXT_LOCALE`	Svuotare il cookie o usare lo switcher (non l'URL)
L'email di conferma non arriva	In questa fase l'autoconferma è attiva: l'account è già utilizzabile	Nessuna azione: arriverà con Resend

Il manuale PDF

Questo documento **si genera**, non si scrive a mano:

- i contenuti sono file Markdown in `docs/manuale/` (uno per capitolo/macro-area);
- `docs/manuale/meta.json` contiene titolo, versione, data e quali fasi copre;
- `npm run manuale` costruisce il PDF con il Chrome installato sul computer (nessun servizio esterno, nessun browser da scaricare) e lo salva come `docs/manuale/OpenStage-Manuale.pdf`;
- l'indice si costruisce **da solo** dai titoli dei file `.md`.

Per aggiornarlo: si modificano i `.md`, si aggiornano versione e data in `meta.json`, si rilancia `npm run manuale` e si committa il PDF. Il generatore è `scripts/manuale.mjs` e **non** entra né nella build di Next.js né nel deploy: serve solo in locale.

Roadmap delle fasi

Fase	Contenuto previsto
1	Fondamenta: design system, accesso, ruoli, pagine pubbliche, database, deploy
1.2	Indurimento del database (sicurezza, indici, avvisi)
1.3	Regole definitive sui ruoli (database, pagine, interfaccia)
2	Mappa (MapLibre + MapTiler) ed eventi reali
3	Pubblicazione e gestione degli eventi, profili artista e locale completi
4	Ricerca, filtri e pagina evento con anteprima social
5	Preferiti e community
6	Email transazionali (Resend) e notifiche
7	Rivendicazione dei locali e inserimento eventi non reclamati
8	Funzioni opzionali per chi vuole sostenere il progetto (PayPal)
9	Rifinitura: prestazioni, accessibilità, contenuti

> Le fasi dalla 3 in poi sono indicative: si dettagliano una per volta, con un piano scritto prima di scrivere codice.

Diario delle fasi

Questa è la memoria del progetto: per ogni fase, cosa è stato costruito, quali decisioni sono state prese e perché, cosa è stato rimandato. Serve per non dover ricostruire a posteriori il ragionamento dietro una scelta.

Fase 1 — Fondamenta

Obiettivo: mettere in piedi la piattaforma "vera" fin dal primo giorno: database reale, accesso reale, niente dati finti o schermate di esempio.

Cosa è stato costruito

- Progetto Next.js 15 (App Router, TypeScript) con Tailwind v4, tutte le pagine pubbliche renderizzate lato server.
- **Design system** dark cyber-neon: sfondo `#07070D`, accenti viola/ciano/rosa, font Space Grotesk + Inter, navbar e footer in vetro, card con gradiente.
- **Doppia lingua IT/EN** con dizionari modulari, scelta automatica in base al paese e selettore manuale ricordato nel cookie `NEXT_LOCALE`.
- **Accesso reale** con Supabase: registrazione email + password, accesso con Google, captcha Cloudflare Turnstile, gestione della sessione con cookie (`@supabase/ssr`) rinnovata dal middleware.
- **Ruoli** (fan / artista / locale / admin) scelti in registrazione e memorizzati nel profilo.
- Pagine: Home, Artisti, Locali, Per gli Artisti, Per le Venue, Chi siamo, Contatti, Privacy, Termini, Cookie, più l'elenco di artisti e locali **letto dal database** con paginazione.
- **Player Spotify** fisso in basso e dock di navigazione mobile.
- Prima migrazione SQL (`0001_init.sql`) con tabelle, RLS e trigger.
- Repository GitHub + deploy continuo su Vercel.

Decisioni e perché

- **Supabase scelto** rispetto a Neon+Clerk, Nhost, Appwrite, Firebase e Turso: è l'unico che unisce Postgres puro e Row Level Security legata all'utente loggato.
- **La scelta del ruolo sta dopo l'accesso con Google**, non nell'URL di ritorno: passare `?role=` avrebbe significato fidarsi di un parametro che il browser può cambiare.
- **Conferma email disattivata** in questa fase, per poter provare subito i flussi senza un servizio di invio email già configurato.
- Elenchi con **paginazione a 12 elementi** e query fatte sul database (nessun dato di esempio).

Verifiche superate: build di produzione senza errori; pagine pubbliche 200 in produzione; i percorsi protetti rimandano al login; ruoli e profili verificati via SQL.

Rimandato: mappa, eventi reali, schede artista/locale complete, ricerca, email.

Fase 1.2 — Indurimento del database

Obiettivo: sistemare il database *prima* che contenga dati veri, quando correggere costa poco.

Cosa è stato costruito (`0003_hardening.sql`)

- `search_path` fissato sulla funzione che aggiorna `updated_at` (evita di eseguire funzioni sbagliate per errore).
- Revoca dell'esecuzione diretta delle funzioni interne di registrazione e di controllo ruolo.
- **3 indici sulle chiavi esterne** di `events` (le ricerche future ne hanno bisogno).
- **12 policy riscritte** con `(select auth.uid())`, che Postgres valuta una volta sola per query invece che riga per riga: è il modo corretto e più veloce.
- **Policy duplicate unite** su `profiles` (da 18 a 16 policy totali).
- **Storico migrazioni regolarizzato**: le voci `0001` … `0003` allineate ai nomi dei file, così gli strumenti Supabase e il repository raccontano la stessa storia.

Verifiche superate: il trigger di registrazione crea ancora il profilo; l'utente può cambiare il proprio ruolo ma non promuoversi ad admin; un utente normale vede una sola riga di `profiles`; i visitatori anonimi non vedono profili né impostazioni; gli avvisi di performance (`auth_rls_initplan`, indici mancanti, policy multiple) sono spariti.

Rimandato: attivazione della protezione "password compromesse" (richiede un intervento in dashboard) e la revoca di alcuni token di servizio usati durante la configurazione.

Fase 1.3 — Ruoli definitivi

Obiettivo: una volta scelto, il ruolo non si cambia più; solo un fan può diventare artista o locale, una volta sola.

Cosa è stato costruito (`0004_role_lock.sql` + interfaccia)

- **Nuova regola nel database**: l'unica transizione consentita è `fan → artista | locale`. Da artista o locale non si torna indietro, e non ci si promuove ad admin.
- **La RLS chiede il ruolo coerente**: per creare una riga `artists` bisogna essere artista, per `venues` bisogna essere locale, per un evento bisogna essere artista o locale. Un fan che prova a farlo via API viene **respinto dal database**, non solo nascosto dall'interfaccia.
- **Correzione di un difetto ereditato dalla Fase 1**: il trigger annullava anche l'assegnazione del ruolo `admin` al profilo dello staff (durante la registrazione non esistono i dati di sessione). Ora funziona e l'account dello staff è allineato.
- **Interfaccia**: badge colorato del ruolo nella barra in alto, link contestuale ("Diventa artista o locale" per un fan, "Crea evento" per artista/locale), pagina di scelta del ruolo riservata ai fan (chi è già artista o locale vede "Il tuo ruolo è già attivo"), pagine protette per ruolo.
- **Pulsante di scelta più onesto**: avvisa che la scelta di Artista/Locale è definitiva e, dopo il salvataggio, **rilegge dal database** il ruolo effettivo: se il database non l'ha accettato, l'utente vede un errore invece di una finta riuscita.
- **Testo corretto**: il sottotitolo della scelta del ruolo diceva "potrai cambiarlo quando vuoi", che con le nuove regole era falso.
- Helper riutilizzabili: `lib/auth/roles.ts` (permessi) e `lib/supabase/session.ts` (utente + ruolo in una sola lettura per richiesta).

Verifiche superate: artista → locale/fan/admin annullati; locale → fan annullato; fan → artista e fan → locale riusciti e poi bloccati; fan respinto (errore 42501) quando prova a creare righe artista o eventi; artista autorizzato; correzione da parte dell'admin possibile; build e produzione verdi; nessun peggioramento negli avvisi di sicurezza.

Rimandato: nulla di bloccante. Le regole verranno estese alle future tabelle (per esempio ai preferiti) man mano che nascono.

Come si aggiorna questo diario

A ogni fine fase si aggiunge una sezione con lo stesso schema: obiettivo, cosa è stato costruito, decisioni e perché, verifiche superate, cosa è rimandato. La review di fase (`docs/review-fase-N.md`) resta il posto dove finiscono giudizi, suggerimenti e debito tecnico da riprendere: qui va solo ciò che è stato deciso e fatto.

Changelog del manuale

Versione	Data	Fasi coperte	Novità
1.0	16 ottobre 2026	1 · 1.2 · 1.3	Prima edizione: guida utente, manuale tecnico (stack, database, sicurezza, deploy, errori noti), diario delle fasi

Come si aggiorna

1. Modifica o aggiungi i file `.md` in `docs/manuale/` (l'ordine dei capitoli segue il nome del file: `01-`, `02-`, …).
2. Aggiorna `versione` e `data` in `docs/manuale/meta.json` e aggiungi una riga a questa tabella.
3. Esegui `npm run manuale`: il PDF `docs/manuale/OpenStage-Manuale.pdf` viene rigenerato con indice e numeri di pagina aggiornati.
4. Committa i sorgenti **e** il PDF.